

Tasco Maps AI Hackathon API Documentation

Reusable API and mock SDK pack for AI map search, autocomplete, POI enrichment, geocoding, nearby discovery, and routing.

Field	Value
Version	2026-06-25
Mock base URL	http://localhost:8787
Recommended facade	https://hackathon.example.com/v1
Current geocode base	https://tasco-maps.dnpwater.vn/geocode
Current route base	https://tasco-maps.dnpwater.vn/route

Version: 2026-06-25

This document defines the API surface that hackathon teams should target when building AI map search, autocomplete, POI enrichment, nearby discovery, geocoding, reverse geocoding, conversational map, or semantic ranking solutions for Tasco Maps.

The goal is pragmatic reuse: a solution should be callable from the current Flutter app through a thin client adapter and should return stable place and route DTOs.

Current App Integration Points

The current app already uses these map service boundaries:

- Search/autocomplete/reverse: `PeliasClient`

(`packages/tasco_maps_core/lib/src/services/pelias_client.dart`)

- Routing: `ValhallaClient`

(`packages/tasco_maps_core/lib/src/services/valhalla_client.dart`)

- Route facade: `RoutingClient`

(`lib/core/core_routing/routing_client.dart`)

- App-compatible search model: `SearchSuggestion`

(`packages/tasco_maps_core/lib/src/models.dart`)

Existing production references:

- Geocode/search base: `https://tasco-maps.dnpwater.vn/geocode`
- Route base: `https://tasco-maps.dnpwater.vn/route`
- Hackathon mock base: `http://localhost:8787`

Recommended hackathon facade base:

`https://hackathon.example.com/v1`

The mock server also supports short aliases such as `/search` and `/route` for quick demos.

Authentication

Hackathon mock APIs accept requests with or without authentication.

Production-ready submissions must support one of these pluggable strategies:

- Authorization: `Bearer <access_token>`
- X-API-Key: `<api_key>`

Do not hardcode credentials in mobile code. The current app has an auth-ready network layer for backend APIs, while the existing map clients are still simple HTTP clients. Keep the map SDK constructor configurable with `baseUrl`, `bearerToken`, or a header provider.

Recommended common headers:

```
Authorization: Bearer <access_token>
X-Request-Id: 3b1d0fb1-7a45-4d2c-9f57-9c0fd85a9b9d
X-Locale: vi-VN
X-Timezone: Asia/Ho_Chi_Minh
```

Common DTOs

PlaceResult

```
{
  "id": "poi:landmark-72",
  "type": "poi",
  "name": "Landmark 72",
  "label": "Landmark 72",
  "address": "Pham Hung, Nam Tu Liem, Ha Noi",
  "category": "office",
  "coordinates": { "lat": 21.0166, "lon": 105.7833 },
  "distanceMeters": 420,
  "score": 0.97,
```

```
"source": "mock",
"tags": ["building", "landmark"]
}
```

Mapping to the app:

- id -> SearchSuggestion.id
- label or name -> SearchSuggestion.label
- category or type -> SearchSuggestion.meta
- address -> SearchSuggestion.description
- coordinates.lat/lon -> SearchSuggestion.coordinates

ErrorResponse

```
{
  "error": {
    "code": "invalid_request",
    "message": "q is required",
    "details": { "field": "q" }
  },
  "requestId": "3b1d0fb1-7a45-4d2c-9f57-9c0fd85a9b9d"
}
```

Common error codes:

HTTP	Code	Meaning
400	invalid_request	Missing or invalid parameter/body
401	unauthorized	Missing or invalid token/key
403	forbidden	Caller is authenticated but not allowed
404	not_found	Resource not found
408	timeout	Upstream/service timeout
429	rate_limited	Too many requests
500	internal_error	Unexpected service error
503	service_unavailable	Upstream unavailable

1. Search API

Free-text search for places, addresses, roads, categories, or coordinates.

Endpoint:

GET /v1/search

Aliases:

GET /search

GET /v1/geocode-search

Request example:

GET /v1/search?q=coffee&lat=21.0278&lon=105.8342&limit=5&lang=vi

Authorization: Bearer <token>

Supported parameters:

Name	Type	Required	Description
q	string	yes	User query
lat	number	no	Focus latitude for local ranking
lon	number	no	Focus longitude for local ranking
radiusMeters	number	no	Ranking/search radius around focus point
bbox	string	no	minLon,minLat,maxLon,maxLat
category	string	no	Optional category filter
limit	integer	no	Default 10, max recommended 20
lang	string	no	Default vi

Response example:

```
{
  "query": "coffee",
  "results": [
    {
      "id": "poi:coffee-house",
      "type": "poi",
      "name": "The Coffee House",
      "label": "The Coffee House",
      "address": "Thai Ha, Dong Da, Ha Noi",
      "category": "cafe",
      "coordinates": { "lat": 21.0129, "lon": 105.8194 },
      "distanceMeters": 1800,
      "score": 0.93,
      "source": "mock"
    }
  ],
  "meta": { "limit": 5, "lang": "vi" }
}
```

Error codes: invalid_request, unauthorized, rate_limited, service_unavailable, internal_error.

2. Autocomplete API

Low-latency suggestions while the user types.

Endpoint:

GET /v1/autocomplete

Alias:

GET /autocomplete

Request example:

GET /v1/autocomplete?q=land&lat=21.0278&lon=105.8342&limit=5&sessionId=s-123

Supported parameters:

Name	Type	Required	Description
q	string	yes	Partial query
lat	number	no	Focus latitude
lon	number	no	Focus longitude
limit	integer	no	Default 5, max recommended 10
sessionId	string	no	Groups keystrokes for ranking/analytics
lang	string	no	Default vi

Response example:

```
{
  "query": "land",
  "suggestions": [
    {
      "id": "poi:landmark-72",
      "type": "poi",
      "name": "Landmark 72",
      "label": "Landmark 72",
      "address": "Pham Hung, Nam Tu Liem, Ha Noi",
      "category": "landmark",
      "coordinates": { "lat": 21.0166, "lon": 105.7833 },
      "score": 0.98,
      "source": "mock"
    }
  ],
  "meta": { "limit": 5, "sessionId": "s-123" }
}
```

Error codes: invalid_request, unauthorized, rate_limited, service_unavailable, internal_error.

3. POI API

Details and enrichment for a selected place.

Endpoint:

```
GET /v1/poi/{id}
```

Alias:

```
GET /poi/{id}
```

Request example:

```
GET /v1/poi/poi:landmark-72
```

Supported parameters:

Name	Type	Required	Description
id	string	yes	POI ID from search/autocomplete/nearby
lang	string	no	Default vi
include	string	no	Comma list: reviews, photos, hours, ai_summary

Response example:

```
{
  "poi": {
    "id": "poi:landmark-72",
    "type": "poi",
    "name": "Landmark 72",
    "label": "Landmark 72",
    "address": "Pham Hung, Nam Tu Liem, Ha Noi",
    "category": "landmark",
    "coordinates": { "lat": 21.0166, "lon": 105.7833 },
    "rating": 4.5,
    "openingHours": "09:00-22:00",
    "aiSummary": "Large mixed-use landmark complex in Nam Tu Liem.",
    "source": "mock"
  }
}
```

Error codes: invalid_request, unauthorized, not_found, service_unavailable, internal_error.

4. Reverse Geocoding API

Resolve coordinates to the nearest address or place.

Endpoint:

GET /v1/reverse-geocoding

Aliases:

GET /reverse-geocoding
GET /v1/reverse

Request example:

GET /v1/reverse-geocoding?lat=21.0166&lon=105.7833&lang=vi

Pelias-compatible request shape also allowed:

GET /v1/reverse?point.lat=21.0166&point.lon=105.7833&lang=vi

Supported parameters:

Name	Type	Required	Description
lat / point.lat	number	yes	Latitude
lon / point.lon	number	yes	Longitude
radiusMeters	number	no	Max lookup radius
lang	string	no	Default vi

Response example:

```
{
  "results": [
    {
      "id": "address:pham-hung",
      "type": "address",
      "name": "Pham Hung",
      "label": "Pham Hung, Nam Tu Liem",
      "address": "Pham Hung, Nam Tu Liem, Ha Noi",
      "category": "address",
      "coordinates": { "lat": 21.0166, "lon": 105.7833 },
      "distanceMeters": 12,
      "source": "mock"
    }
  ]
}
```

Error codes: invalid_request, unauthorized, not_found, service_unavailable, internal_error.

5. Nearby Search API

Find nearby places around a point.

Endpoint:

GET /v1/nearby-search

Alias:

GET /nearby-search

Request example:

GET /v1/nearby-search?lat=21.0278&lon=105.8342&radiusMeters=1500&category=restaurant&limit=10

Supported parameters:

Name	Type	Required	Description
lat	number	yes	Center latitude
lon	number	yes	Center longitude
radiusMeters	number	no	Default 1000, max recommended 5000
category	string	no	Example: hotel, restaurant, cafe, parking
openNow	boolean	no	Filter by opening status if known
limit	integer	no	Default 10, max recommended 20
lang	string	no	Default vi

Response example:

```
{
  "center": { "lat": 21.0278, "lon": 105.8342 },
  "results": [
    {
      "id": "poi:hoan-kiem-lake",
      "type": "poi",
      "name": "Hoan Kiem Lake",
      "label": "Hoan Kiem Lake",
      "address": "Hoan Kiem, Ha Noi",
      "category": "landmark",
      "coordinates": { "lat": 21.0287, "lon": 105.8521 },
      "distanceMeters": 950,
      "score": 0.91,
      "source": "mock"
    }
  ],
  "meta": { "radiusMeters": 1500, "limit": 10 }
}
```

Error codes: invalid_request, unauthorized, rate_limited, service_unavailable, internal_error.

6. Geocoding API

Resolve structured or natural-language addresses to coordinates.

Endpoint:

GET /v1/geocoding

Alias:

GET /geocoding

Request example:

GET /v1/geocoding?address=Pham%20Hung%20Nam%20Tu%20Liem%20Ha%20Noi&limit=5

Supported parameters:

Name	Type	Required	Description
address	string	yes	Address text
city	string	no	City/province hint
district	string	no	District hint
lat	number	no	Focus latitude for disambiguation
lon	number	no	Focus longitude for disambiguation
limit	integer	no	Default 5, max recommended 10
lang	string	no	Default vi

Response example:

```
{
  "query": "Pham Hung Nam Tu Liem Ha Noi",
  "results": [
    {
      "id": "address:pham-hung",
      "type": "address",
      "name": "Pham Hung",
      "label": "Pham Hung, Nam Tu Liem",
      "address": "Pham Hung, Nam Tu Liem, Ha Noi",
      "category": "address",
      "coordinates": { "lat": 21.0166, "lon": 105.7833 },
      "score": 0.88,
      "source": "mock"
    }
  ]
}
```

Error codes: invalid_request, unauthorized, rate_limited, service_unavailable, internal_error.

7. Route API

Calculate primary and alternate routes. The app currently integrates routing through a Valhalla-backed `RoutingClient`. A hackathon solution can either return Valhalla-compatible payloads or the normalized route DTO below and ship a thin adapter.

Endpoint:

POST /v1/route

Alias:

POST /route

Request example:

POST /v1/route

Content-Type: application/json

```
{
  "locations": [
    { "lat": 21.0278, "lon": 105.8342 },
    { "lat": 21.0166, "lon": 105.7833 }
  ],
  "mode": "auto",
  "alternates": 2,
  "language": "vi-VN",
  "units": "kilometers"
}
```

Supported body fields:

Name	Type	Required	Description
locations	array	yes	At least origin and destination
locations[].lat	number	yes	Latitude
locations[].lon	number	yes	Longitude
mode	string	no	auto, pedestrian, or bicycle; default auto
alternates	integer	no	Number of alternatives, default 2
language	string	no	Default vi-VN
units	string	no	Default kilometers
avoidTolls	boolean	no	Optional routing preference
avoidHighways	boolean	no	Optional routing preference

Response example:

```
{
  "routes": [
    {
      "routeId": "route:primary",
      "sourceIndex": 0,
      "summary": {
        "distanceMeters": 6800,
        "durationSeconds": 950
      },
      "geometry": {
        "type": "LineString",
        "coordinates": [
          [105.8342, 21.0278],
          [105.812, 21.022],
          [105.7833, 21.0166]
        ]
      },
      "maneuvers": [
        {
          "instruction": "Đi về phía Tây theo đường chính.",
          "distanceMeters": 2300,
          "durationSeconds": 320,
          "beginShapeIndex": 0,
          "endShapeIndex": 1,
          "streetNames": ["Tran Duy Hung"]
        }
      ]
    }
  ],
  "meta": { "mode": "auto", "alternates": 2 }
}
```

Error codes: `invalid_request`, `unauthorized`, `not_found`, `timeout`, `service_unavailable`, `internal_error`.

Mock Server

Run:

```
node docs/hackathon/mock_api_server.js
```

Health check:

```
curl "http://localhost:8787/health"
```

Search example requested by the organizer:

```
curl "http://localhost:8787/search?q=coffee"
```

Expected shape:

```
{
  "query": "coffee",
  "results": [
    {
      "id": "poi:coffee-house",
      "label": "The Coffee House",
      "coordinates": { "lat": 21.0129, "lon": 105.8194 }
    }
  ]
}
```

Submission Expectations For Hackathon Teams

Teams should submit:

- API endpoint or deployable service.
- OpenAPI/Swagger spec or equivalent contract.
- SDK/client adapter for Flutter/Dart or REST examples.
- Mock/staging endpoint and sample data.
- Notes about ranking/enrichment logic, latency, fallback behavior, and data

provenance.

Compatibility requirements:

- Return stable IDs.
- Keep coordinates as WGS84 latitude/longitude.
- Preserve Vietnamese text and diacritics in real services.
- Support configurable base URL and authentication.
- Avoid app-specific UI dependencies in the service/SDK layer.
- Provide deterministic mock data for tests and demos.